

TP — Migrer StayBook vers Kubernetes avec Argo CD (GitOps)



Sujet étudiant — TP semi-guidé

Enseignant : Youssef YOUNES

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

Modalité : individuel ou binôme, sur votre machine (macOS, Linux ou Windows + WSL2)

Prérequis : Docker, kind, kubectl, git, un compte GitHub, et l'archive du TP (sources StayBook + dépôt staybook-gitops)

Objectif : partir de l'application StayBook telle que vous l'avez construite avec Docker Compose, et la faire tourner dans un cluster Kubernetes **piloté par Git** grâce à Argo CD. Les manifests Kubernetes sont fournis : votre travail porte sur Argo CD et sur la boucle GitOps.

Comment travailler sur ce TP

Ce TP est **semi-guidé**. Les commandes vous sont données dans l'ordre, mais à chaque étape des **questions** vous demandent d'observer, de chercher dans la documentation ou d'expliquer ce qui vient de se passer. C'est en répondant que vous apprenez : ne vous contentez pas de copier-coller.

- Lisez la sortie de chaque commande avant de passer à la suivante.
- Notez vos réponses dans le fichier `REPONSES.md` fourni dans l'archive, que vous commiterez dans votre dépôt.
- Documentation officielle : <https://argo-cd.readthedocs.io>. Apprenez à y chercher.
- Bloqué plus de dix minutes ? Demandez.

Ce que contient l'archive

TP-ArgoCD-StayBook/	
├─ staybook-src/	sources StayBook (TP Docker) + docker-compose.yml
└─ staybook-gitops/	le futur dépôt Git "source de vérité"
├─ kind-config.yaml	cluster kind + redirections de ports
├─ build-images.sh	construit les 5 images, les charge dans kind
├─ argocd-install/	Argo CD : manifests officiels + patches
├─ apps/staybook/	2 PostgreSQL, hotels, reservations, gateway, front
├─ apps/concierge/	Ollama, Qdrant, rag-service (l'IA métier)
├─ argocd/	les Applications qui relient dépôt et cluster
└─ REPONSES.md	votre fiche de réponses

Étape 1 — Docker Compose : le point de départ

Partie 0 — StayBook en Docker Compose

Vous connaissez StayBook : un site de réservation d'hôtels composé de 9 conteneurs, dont un **concierge IA** (Ollama + Qdrant + un service RAG). Dans le TP Docker, vous l'avez décrit dans un `docker-compose.yml`. Relancez-le une dernière fois, pour avoir un point de comparaison.

```
cd staybook-src
docker compose up -d --build
docker compose ps
```

Ouvrez <http://localhost:8080>, vérifiez la liste des hôtels, puis posez une question au concierge (la première réponse est lente : Ollama télécharge ses modèles).

Question 1. Dans ce `docker-compose.yml`, repérez tout ce que Compose fait **pour vous** et que vous n'aurez plus « gratuitement » dans Kubernetes : l'ordre de démarrage, le réseau, les volumes, les ports, les variables d'environnement, la construction des images. Pour chacun, notez le mot-clé Compose correspondant.

Puis arrêtez tout, en **gardant les images** :

```
docker compose down
docker images | grep staybook
```

Question 2. Pourquoi faut-il impérativement faire `docker compose down` avant de continuer ? (Deux raisons : l'une concerne un port, l'autre la mémoire.)

Étape 2 — Kubernetes + Argo CD : la migration

Partie 1 — Avant de taper une commande : GitOps et Argo CD

1.1 Push ou pull ?

Jusqu'ici, pour déployer, vous faisiez `docker compose up` ou `kubectl apply`. Dans les entreprises, c'est souvent un pipeline CI (GitLab CI, GitHub Actions, Jenkins) qui le fait, avec un accès administrateur au cluster. C'est le modèle **push** : quelqu'un pousse la configuration vers le cluster.

GitOps renverse la logique : on décrit l'état voulu dans un dépôt Git, et **un agent installé dans le cluster** vient lire ce dépôt et fait converger le cluster vers ce qui y est décrit. C'est le modèle **pull**. Argo CD est cet agent.

Les quatre principes GitOps (groupe OpenGitOps, CNCF) :

1. **Déclaratif** : le système est décrit (manifests), pas scripté.
2. **Versionné et immuable** : l'état désiré est dans Git, avec son historique.
3. **Tiré automatiquement** : l'agent va chercher l'état désiré, on ne pousse pas vers le cluster.
4. **Réconcilié en continu** : l'agent compare en permanence l'état réel à l'état désiré et corrige.

Question 3. En mode push, qui détient les droits d'administration sur le cluster ? Citez deux risques concrets que le modèle pull réduit.

Question 4. Un collègue modifie « à la main » le nombre de réplicas d'un déploiement avec `kubectl scale` sur un cluster géré par Argo CD. D'après le principe 4, que va-t-il se passer ? Est-ce souhaitable ?

1.2 Les composants d'Argo CD

Argo CD n'est pas un seul programme. Lisez la page « Architectural Overview » de la documentation et complétez le tableau (vous le vérifierez en partie 4).

Composant	Rôle (une phrase)	Deployment ou StatefulSet ?
argocd-server		
argocd-repo-server		
argocd-application-controller		
argocd-redis		
argocd-dex-server		
argocd-applicationset-controller		
argocd-notifications-controller		

Question 5. Quel composant est le seul à parler à GitHub ? Lequel est le seul à créer des ressources dans le cluster ?

Partie 2 — Un cluster Kubernetes vierge avec kind

2.1 Vérifier l'outillage

Outil	Vérification	Version minimale
Docker	<code>docker info</code> répond sans erreur	24
kind	<code>kind version</code>	0.20
kubectl	<code>kubectl version --client</code>	1.28
git	<code>git --version</code>	2.30

Installation : <https://kind.sigs.k8s.io/docs/user/quick-start/#installation>.

2.2 Créer le cluster

kind fait tourner chaque nœud Kubernetes dans un conteneur Docker. Un service `NodePort` écoute donc **dans le conteneur**, pas sur votre machine. Le fichier `kind-config.yaml` fourni déclare les redirections nécessaires :

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: argocd-tp
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080 # Argo CD (interface web)
    hostPort: 8080
  - containerPort: 30081 # StayBook (front)
    hostPort: 8081
```

```
cd staybook-gitops
kind create cluster --config kind-config.yaml
kubectl config current-context
kubectl get nodes
kubectl get pods -A
```

Question 6. Le cluster est « vierge ». Quels namespaces existent déjà, et à quoi servent-ils ? Combien de pods tournent, et pourquoi n'y en a-t-il aucun dans `default` ?

Partie 3 — Mettre les images StayBook dans le cluster

Dans Compose, `build:` construisait les images et elles étaient immédiatement utilisables. Dans Kubernetes, chaque nœud doit **tirer** les images depuis un registre... que nous n'avons pas. kind offre un raccourci : charger une image locale directement dans les nœuds.

```
./build-images.sh ../staybook-src v1
```

Le script construit `staybook-front:v1`, `staybook-gateway:v1`, `staybook-hotels-service:v1`, `staybook-reservations-service:v1`, `staybook-rag-service:v1`, puis les charge avec `kind load docker-image`. Lisez-le, il tient en quinze lignes.

Question 7. Pourquoi le script tague les images `v1` plutôt que `latest` ? Cherchez la règle de Kubernetes sur `imagePullPolicy` quand le tag est `latest` : que se passerait-il au démarrage d'un pod ?

Question 8. Où sont physiquement les images maintenant ? Vérifiez avec `docker exec argocd-tp-control-plane crictl images | grep staybook`. Qu'est-ce que `crictl`, et pourquoi n'est-ce pas `docker images` ?

Partie 4 — Installer Argo CD

4.1 Lire avant d'appliquer

Argo CD s'installe avec un seul fichier officiel, `install.yaml`. Mais nous voulons deux ajustements : exposer l'interface en **NodePort 30080** (celui redirigé par `kind`) et accepter le **HTTP** (pas de certificat en local). Plutôt que de modifier le fichier officiel ou de « patcher » à la main après coup, le dossier `argocd-install/` utilise **Kustomize**, intégré à `kubectl` :

```
argocd-install/
├─ kustomization.yaml      la recette : le fichier officiel + nos patches
├─ namespace.yaml         le namespace argocd
├─ argocd-server-nodeport.yaml patch : Service en NodePort 30080
└─ argocd-cmd-params.yaml  patch : server.insecure = true
```

Ouvrez `kustomization.yaml` et regardez ce que Kustomize va produire, **sans rien appliquer** :

```
kubectl kustomize argocd-install | grep -c '^kind:'
kubectl kustomize argocd-install | grep '^kind:' | sort | uniq -c
```

Question 9. Combien d'objets vont être créés ? Parmi les types listés, trois ne sont pas des objets Kubernetes « standard ». Lesquels, et à quoi servent-ils ? (Indice : `CustomResourceDefinition`.)

Question 10. Pourquoi est-il préférable de passer par Kustomize plutôt que de faire `kubectl apply -f install.yaml` puis `kubectl patch` sur le Service ? Pensez au jour où vous mettrez Argo CD à jour.

4.2 Installer

```
kubectl apply -k argocd-install --server-side
kubectl get pods -n argocd -w          # Ctrl+C quand tout est Running
kubectl wait --for=condition=Ready pods --all -n argocd --timeout=300s
```

```
kubectl get deploy,sts -n argocd
kubectl get crd | grep argoproj
```

Question 11. Vérifiez votre tableau de la partie 1 : quel composant est un StatefulSet ? Avez-vous les sept composants ?

Question 12. Essayez `kubectl apply -k argocd-install sans --server-side` (c'est sans danger, tout existe déjà). Quelle erreur obtenez-vous, sur quel objet ? Cherchez la cause (`last-applied-configuration`).

4.3 Se connecter à l'interface

Grâce au patch NodePort, l'interface est déjà sur <http://localhost:8080>. Le mot de passe initial de `admin` est dans un Secret :

```
kubectl get svc argocd-server -n argocd
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d; echo
```

Connectez-vous. L'interface est vide : aucune application. Normal.

Question 13. Pourquoi la commande se termine-t-elle par `| base64 -d` ? Que verriez-vous sans ? Un Secret Kubernetes est-il chiffré ?

4.4 Installer et utiliser la CLI

L'interface sert à visualiser. Tout ce que l'on fera de « sérieux » passera par la CLI ou par du YAML.

```
# macOS
brew install argocd
# Linux
curl -sSL -o argocd https://github.com/argoproj/argo-cd/releases/latest/download/
argocd-linux-amd64
sudo install -m 555 argocd /usr/local/bin/argocd
# Windows : argocd-windows-amd64.exe depuis la page Releases, à renommer argocd.exe
```

```
argocd login localhost:8080 --username admin --plaintext --grpc-web
argocd version
argocd app list
argocd account update-password
```

Question 14. Que signifient `--plaintext` et `--grpc-web` ? Que se passe-t-il sans `--plaintext` (essayez) ? Ces options seraient-elles acceptables en production ?

Partie 5 — Relier Argo CD à votre compte GitHub

5.1 Créer le dépôt « source de vérité »

Sur GitHub, créez un dépôt **privé** nommé `staybook-gitops` (vide, sans README). Puis, depuis le dossier `staybook-gitops` de l'archive :

```
git init -b main
git add .
git commit -m "feat: StayBook manifests + Argo CD install (v1)"
git remote add origin https://github.com/<votre-login>/staybook-gitops.git
git push -u origin main
```

Vérifiez sur GitHub que les dossiers `apps/`, `argocd/` et `argocd-install/` sont bien là.

Question 15. Regardez `apps/staybook/00-secrets.yaml` : les mots de passe PostgreSQL sont en clair dans Git. Est-ce acceptable ici ? Et en entreprise ? Citez une solution.

5.2 Déclarer le dépôt dans Argo CD

Essayez d'abord **sans** identifiants :

```
argocd repo add https://github.com/<votre-login>/staybook-gitops.git
```

Question 16. Notez l'erreur. Pourquoi GitHub répond-il « Repository not found » et non « accès refusé » ?

Créez un **token** : GitHub → Settings → Developer settings → Personal access tokens → **Fine-grained tokens** → Generate. Limitez-le au dépôt `staybook-gitops`, avec la seule permission **Contents : Read-only**. Puis :

```
argocd repo add https://github.com/<votre-login>/staybook-gitops.git \
  --username <votre-login> --password <le-token>
argocd repo list
```

Le statut doit être `Successful`. Allez voir dans l'UI : Settings → Repositories.

Question 17. Sous quelle forme Argo CD a-t-il stocké ces identifiants dans le cluster ? Trouvez l'objet avec `kubectl get secrets -n argocd -l argocd.argoproj.io/secret-type=repository` et listez ses clés (`-o jsonpath='{.data}'` puis décidez `url` et `username`, pas `password`).

Question 18. Comparez trois façons de s'authentifier (token personnel, deploy key SSH, GitHub App) : portée, rotation, ce qui se passe quand la personne quitte l'entreprise. Laquelle recommanderiez-vous pour 200 dépôts ?

Partie 6 — Déployer StayBook : l'Application `staybook`

Une **Application** Argo CD relie une **source** (dépôt + chemin + révision) à une **destination** (cluster + namespace). Vous allez la créer de deux façons.

6.1 Par l'interface, pour voir

+ New App , puis :

Champ	Valeur
Application Name	<code>staybook</code>
Project	<code>default</code>
Sync Policy	<code>Manual</code>
Repository URL	votre dépôt
Revision	<code>HEAD</code>
Path	<code>apps/staybook</code>
Cluster URL	<code>https://kubernetes.default.svc</code>
Namespace	<code>staybook</code>
Sync Options	cocher <code>Auto-create namespace</code>

Créez. L'application apparaît **OutOfSync / Missing** : Argo CD a comparé Git au cluster, a vu que rien n'existe, et **attend votre ordre**. Avant de synchroniser, cliquez sur l'application : vous voyez l'arbre des ressources que Git décrit.

Question 19. Combien de ressources Argo CD compte-t-il déployer ? Retrouvez chacune dans les fichiers de `apps/staybook/`.

Cliquez `Sync` → `Synchronize`, et **regardez l'ordre** dans lequel les ressources passent au vert. Puis ouvrez <http://localhost:8081>.

Question 20. Les ressources ne sont pas déployées toutes en même temps. Ouvrez les manifests et trouvez l'annotation qui pilote cet ordre. Quel ordre a été choisi, et pourquoi ? À quoi cela correspondait-il dans `docker-compose.yml` ?

Question 21. « OutOfSync » et « Healthy » sont deux statuts indépendants. Donnez un exemple d'application Synced mais Degraded , et un exemple OutOfSync mais Healthy .

6.2 En YAML, pour de vrai

Supprimez l'application (`argocd app delete staybook` , confirmez), vérifiez que le namespace `staybook` s'est vidé. Ouvrez `argocd/staybook-application.yaml` : c'est exactement ce que l'interface a créé pour vous. Remplacez `<VOTRE-LOGIN>` par votre login GitHub, committez, poussez, et appliquez **une fois** :

```
kubectl apply -f argocd/staybook-application.yaml
argocd app list
argocd app sync staybook
argocd app get staybook
```

Question 22. Vous venez de faire un `kubectl apply ...` n'est-ce pas contraire au GitOps ? Pourquoi ce dernier `apply` est-il inévitable, et comment réduire cette « amorce » à un seul objet pour tout un cluster ? (Cherchez « App of Apps ».)

Question 23 — compose → Kubernetes. Reprenez votre réponse à la question 1 et complétez la colonne de droite en vous appuyant sur les manifests fournis.

Ce que faisait Compose	Mot-clé Compose	Dans les manifests Kubernetes
Ordre de démarrage		
Attendre qu'un service soit prêt		
Réseau et DNS entre services		
Persistance des données		
Exposer le site		
Variables d'environnement		
Mots de passe		
Image à utiliser		

Partie 7 — Déployer l'IA métier : l'Application concierge

Le concierge, c'est trois composants : **Ollama** (le serveur de modèles), **Qdrant** (la base vectorielle) et **rag-service** (le code qui indexe les hôtels et construit les réponses). Ils sont dans un dossier séparé, `apps/concierge/` , et auront leur propre Application : on peut ainsi déployer, mettre à jour ou retirer l'IA indépendamment du site.

Complétez `argocd/concierge-application.yaml` (même login), committez, poussez, puis :

```
kubectl apply -f argocd/concierge-application.yaml
argocd app sync concierge
kubectl get pods -n staybook -w
```

Le pod `rag-service` reste un moment en `Init:0/2`, puis `Running`. Suivez ce qu'il fait :

```
kubectl -n staybook logs deploy/rag-service -f
```

Quand vous lisez `Indexed 6 hotels`, posez une question au concierge sur <http://localhost:8081>, ou en ligne de commande :

```
curl -s -X POST http://localhost:8081/api/chat -H 'Content-Type: application/json' \
-d '{"message": "Je cherche un hotel a Paris pour 2 personnes"}'
```

Question 24. Pourquoi les deux Applications pointent-elles vers le **même namespace** `staybook` ? Que se passerait-il si le concierge était dans un namespace `concierge` ? (Indice : regardez `RAG_URL` dans le manifest de la gateway.)

Question 25. Ouvrez `apps/concierge/20-rag-service.yaml`. Que font les deux `initContainers` ? En quoi est-ce différent des sync waves de la question 20 ? Pourquoi avoir les deux ?

Question 26. Ouvrez `apps/concierge/11-ollama.yaml`. Où sont stockés les modèles téléchargés ? Supprimez le pod Ollama (`kubectl -n staybook delete pod -l app=ollama`) et vérifiez avec `kubectl -n staybook exec deploy/ollama -- ollama list` que les modèles sont toujours là. Qu'est-ce qui les a conservés ? Que signifie `strategy: Recreate` ?

Question 27. Toujours dans ce fichier, lisez le bloc `resources`. Que se passe-t-il si Ollama dépasse la limite mémoire ? Et si le nœud n'a pas les 2 Gi demandés ?

Étape 3 — GitOps : Git pilote le cluster

Partie 8 — La boucle GitOps en action

Les deux Applications sont Synced / Healthy. À partir de maintenant, **vous n'avez plus le droit de toucher au cluster avec `kubectl apply`, `edit` ou `scale`** sur StayBook. Tout passe par Git. Sauf, bien sûr, pour jouer le rôle du collègue indiscipliné.

8.1 Un commit, un déploiement

Dans `apps/staybook/20-hotels-service.yaml`, passez `replicas: 1` à `replicas: 2`. Commitez, poussez. Puis :

```
argocd app get staybook --refresh
argocd app diff staybook
argocd app sync staybook
kubectl -n staybook get deploy hotels-service
```

Question 28. Sans `--refresh`, au bout de combien de temps Argo CD aurait-il vu le commit ? D'où vient cette valeur (ConfigMap `argocd-cm`, clé `timeout.reconciliation`) ? Quel mécanisme permettrait de réagir en quelques secondes sans interroger GitHub en boucle, et pourquoi ne peut-on pas l'utiliser ici ?

8.2 Activer la synchronisation automatique, dans Git

Modifiez `argocd/staybook-application.yaml` pour ajouter la politique automatique :

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
```

Commitez, poussez... et constatez que rien ne change dans Argo CD. Pourquoi ? Parce que l'objet `Application` lui-même n'est pas géré par Argo CD (question 22). Appliquez-le une dernière fois à la main :

```
kubectl apply -f argocd/staybook-application.yaml
argocd app get staybook | grep "Sync Policy"
```

Faites la même chose pour `concierge`.

8.3 La dérive et l'auto-réparation

Jouez le collègue indiscipliné, deux terminaux côte à côte :

```
# terminal 1
kubectl -n staybook get deploy front -w
```

```
# terminal 2
kubectl -n staybook scale deploy front --replicas=3
```

Question 29. Que s'est-il passé, et en combien de temps ? Quel composant a agi ? Retrouvez sa trace : `kubectl -n argocd logs sts/argocd-application-controller | grep staybook | tail`. Essayez plus fort : `kubectl -n staybook delete deploy gateway`. Le site est-il resté accessible ?

8.4 Livrer une nouvelle version du front

C'est le cas d'usage quotidien. Modifiez le titre de la page dans `staybook-src/front/public/index.html` (par exemple `StayBook v2`), puis :

```
docker build -t staybook-front:v2 ../staybook-src/front
kind load docker-image --name argocd-tp staybook-front:v2
```

Puis, dans le dépôt GitOps, changez `image: staybook-front:v1` en `v2` dans `apps/staybook/40-front.yaml`. Commitez, poussez, attendez (ou `--refresh`), et rechargez <http://localhost:8081>.

Question 30. Décrivez précisément la chaîne d'événements entre votre `git push` et le nouveau titre dans le navigateur. Qui a fait quoi ? Où est passé l'ancien pod ? (`kubectl -n staybook get rs`)

Question 31. Dans une entreprise, qui construirait et chargerait l'image à votre place, et où ? Quelle partie de ce que vous venez de faire relève de la **CI**, et quelle partie du **CD** ?

8.5 Le nettoyage automatique (prune)

Ajoutez un fichier `apps/staybook/99-test.yaml` contenant un ConfigMap quelconque, commitez, poussez, vérifiez qu'il apparaît dans le cluster. Puis supprimez le fichier (`git rm`), commitez, poussez.

Question 32. Qu'est-il arrivé au ConfigMap ? Que se serait-il passé avec `prune: false` ? Pourquoi `prune` est-il désactivé par défaut dans Argo CD ?

8.6 Revenir en arrière

Faites un « mauvais » déploiement : dans Git, changez l'image du front en `staybook-front:v3` (qui n'existe pas). Poussez. Observez le statut Health et le pod.

Deux façons de revenir en arrière :

```
# (a) la façon Argo CD
argocd app history staybook
argocd app rollback staybook <ID-précédent>
# (b) la façon Git
git revert HEAD && git push
```

Faites (a), lisez le message d'Argo CD, puis faites (b).

Question 33. Après le rollback (a), l'auto-sync est-il toujours actif ? Pourquoi Argo CD fait-il cela ? Laquelle des deux approches est « GitOps-compatible » ? Le site est-il resté disponible pendant le mauvais déploiement, et grâce à quoi ?

Question 34. Reliez chaque entrée de `argocd app history staybook` à un commit de `git log --oneline`. Face à un auditeur qui demande « qui a déployé quoi, quand », que répondez-vous, en combien de commandes ?

Partie 9 — Nettoyage

```
kind delete cluster --name argocd-tp
docker image prune -f # optionnel
```

Votre dépôt GitHub et votre token restent : révoquez le token si vous ne comptez pas refaire le TP.

Livrable

Commitez dans votre dépôt `staybook-gitops` le fichier `REPONSES.md` avec vos réponses aux questions 1 à 34, et donnez à votre enseignant l'accès en lecture au dépôt. L'historique Git de votre dépôt fait partie du livrable : il raconte votre TP.

Annexe A — Aide-mémoire

Besoin	Commande
État des applications	<code>argocd app list</code>
Détail d'une application	<code>argocd app get staybook</code>
Forcer la lecture de Git	<code>argocd app get staybook --refresh</code>
Différence Git / cluster	<code>argocd app diff staybook</code>
Synchroniser	<code>argocd app sync staybook</code>
Historique des syncs	<code>argocd app history staybook</code>
Rollback d'urgence	<code>argocd app rollback staybook <id></code>
Auto-sync en CLI	<code>argocd app set staybook --sync-policy automated --self-heal --auto-prune</code>
Logs du controller	<code>kubectl -n argocd logs sts/argocd-application-controller -f</code>
Logs du repo-server (problèmes Git)	<code>kubectl -n argocd logs deploy/argocd-repo-server -f</code>
Mot de passe initial	<code>kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" base64 -d</code>
Applications en natif	<code>kubectl get applications -n argocd</code>
Images présentes dans le nœud	<code>docker exec argocd-tp-control-plane crictl images</code>
Relancer l'indexation du concierge	<code>curl -X POST http://localhost:8081/api/rag/reindex</code>

Annexe B — Pour aller plus loin : installer Argo CD avec Helm

Variante optionnelle, pour ceux qui ont fini. Helm remplace « fichier officiel + patches Kustomize » par « chart + values » et apporte `helm upgrade` / `helm rollback`.

```
kubectl delete -k argocd-install # on repart propre
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
cat > values-argocd.yaml <<'E0V'
configs:
  params:
    server.insecure: true
server:
  service:
    type: NodePort
    nodePortHttp: 30080
dex:
  enabled: false
```

```
notifications:
  enabled: false
EOV
helm install argocd argo/argo-cd -n argocd --create-namespace --version 10.9.6 -f
values-argocd.yaml
helm list -n argocd
```

Les Applications et le dépôt sont à redéclarer (ils vivaient dans le namespace supprimé). Comparez :
`helm history`, `helm upgrade` après modification des values, `helm rollback argocd 1`.